Universidad Simón Bolívar

Departamento de Computación y Tecnología de la Información

USB - LSBD - PRY03

Especificación del esquema conceptual-lógico-físico

Base de datos - Sistema USB Bank

Plataforma bancaria 100 % digital

Descripción de diseño de software

Estructurado conforme a IEEE Std 1016™-2009

CI3391 - Sistemas de bases de datos Laboratorio - Fase III

Trimestre I - 2026

Profesor: Kendy de J. Briceño B.

Autores:

Pacheco, Ángel - 20-10479
Ramírez, Rosa - 20-10527
García, Ricardo - 20-10274
Orta, Brian - 21-10447
Isea, Luis M. - 19-10175
Valero, Ángel - 18-10436

13 de abril de 2026

Resumen: El presente documento técnico especifica el diseño conceptual, lógico y físico de la base de datos para el sistema USB Bank (Fase III). El esquema consta de 37 tablas normalizadas hasta BCNF y 4FN/5FN, organizadas en módulos de Catálogos, Canales, Clientes/KYC, Productos Financieros, Transacciones y Auditoría. Se implementan especializaciones mediante herencia de claves primarias para CLIENTE, CANAL, CUENTA y TARJETA. Se documentan las demostraciones formales de normalización, la justificación de interoperabilidad para transferencias interbancarias, y la restricción 1:1 de Pago Móvil. El documento incluye anexos extensos con el código fuente SQL y Python implementado. Sigue los lineamientos del estándar IEEE Std 1016-2009.

Palabras clave: 4FN, 5FN, base de datos, descripción de diseño de software, especialización, interoperabilidad bancaria, normalización BCNF, PostgreSQL, USB Bank

© 2026 Universidad Simón Bolívar
Departamento de Computación
Todos los derechos reservados.

Documento: USB - LSBD - PRY03 - TECN - 2010479 - 2010527 - 2010274 - 2110447 - 1910175 - 1810436
Fecha de entrega: 13 de abril de 2026

Índice

1 Introducción **5**
1.1 Propósito del documento 5
1.2 Alcance 5
1.3 Contexto del sistema 6

2 Demostración formal de normalización **6**
2.1 Boyce-Codd Normal Form (BCNF) 6
2.2 Cuarta Forma Normal (4FN) y Quinta Forma Normal (5FN) 7

3 Justificación de interoperabilidad: tabla MOVIMIENTO **9**

4 Restricción matemática 1:1: tabla AFILIACION_PAGOMOVIL **10**

5 Modelo conceptual y lógico **10**
5.1 Diagrama entidad-relación extendido 10
5.2 Esquema entidad relación extendido 11

6 Modelo relacional **11**
6.1 Esquema relacional base 11
6.2 Esquema relacional base 12
6.3 Esquema relacional en notación moderna 13

7 Diccionario de datos físico **13**
7.1 Módulo 1: Catálogos e Identidad 14
7.2 Módulo 2: Canales (Jerarquía de especialización) 15
7.3 Módulo 3: Clientes y KYC 15
7.4 Módulo 4: Productos Financieros 17
7.5 Módulo 5: Transacciones y Auditoría 18

8 Restricciones de integridad **20**
8.1 Integridad de entidad 20
8.2 Integridad referencial 20
8.3 Restricciones de dominio y negocio 20

9 Anexos: Código fuente implementado **20**
9.1 A. Script maestro SQL: creación de tablas completa 20
9.2 B. Script de inserts: extracto representativo 28
9.3 C. Trigger de auditoría (SQL) 29
9.4 D. Función fn_reporte_estadistico completa 29
9.5 E. Función fn_reporte_contable_base completa 31
9.6 F. Extracto de sp_imprimir_reporte_estadistico (RAISE NOTICE) . . 34

9.7 G. Python: conexión PostgreSQL y obtener_datos_y_generar (main.py) 36
9.8 H. Python: extracto clave de generador_reportes.py 39

10 Referencias **41**

1. Introducción

El presente documento técnico especifica el esquema conceptual, lógico y físico de la base de datos para el sistema USB Bank (Fase III). El modelo entidad-relación extendido (EER) resultante constituye la base para la implementación física en PostgreSQL 16+. Se presentan cuatro representaciones principales: el diagrama EER extendido, el esquema relacional base, el modelo relacional en notación moderna (37 tablas), y el diccionario de datos físico detallando tipos, restricciones y justificaciones de diseño.

Este documento sigue los lineamientos del estándar IEEE Std 1016-2009 para descripciones de diseño de software, organizando la información en vistas de diseño que abordan las preocupaciones de los stakeholders técnicos: desarrolladores, administradores de bases de datos y evaluadores académicos.

1.1. Propósito del documento

Comunicar el diseño de la base de datos de USB Bank a stakeholders técnicos, documentando: (1) las demostraciones formales de normalización BCNF/4FN/5FN, (2) la justificación de decisiones de interoperabilidad, (3) el diccionario de datos físico, y (4) los anexos de código fuente para replicabilidad.

1.2. Alcance

El alcance cubre el diseño lógico y físico del sistema USB Bank, incluyendo:

- Demostraciones formales de normalización (BCNF, 4FN, 5FN)
- Diagrama EER extendido
- Esquema relacional base
- Modelo relacional en notación moderna (37 tablas)
- Diccionario de datos físico completo (PostgreSQL)
- Justificación de interoperabilidad para transferencias interbancarias
- Restricción 1:1 para afiliación de Pago Móvil
- Módulos: Catálogos, Canales, Clientes/KYC, Productos, Transacciones, Auditoría
- Anexos con código fuente SQL y Python implementado

1.3. Contexto del sistema

USB Bank es una plataforma bancaria 100 % digital que opera sin sucursales físicas. Todos los procesos se realizan a través de canales digitales (Web, App Móvil) y electrónicos (ATM, IVR, POS). El sistema soporta operaciones para clientes naturales y jurídicos, incluyendo apertura de cuentas, emisión de tarjetas (débito/crédito), transferencias, pagos POS/e-commerce, Pago Móvil y generación de reportes analíticos.

2. Demostración formal de normalización

Esta sección presenta las demostraciones teóricas que justifican el nivel de normalización alcanzado por el esquema de USB Bank, conforme a los requerimientos académicos de la asignatura CI3391.

2.1. Boyce-Codd Normal Form (BCNF)

Una relación R está en BCNF si y solo si, para toda dependencia funcional no trivial $X \to Y$ que se cumple en R, X es una superclave de R.

2.1.1. Extracción de DOCUMENTO_IDENTIDAD

En la versión inicial, la tabla `VALIDACION_KYC` contenía los atributos:

$$\{\text{id_validación, id_cliente, nro_documento, tipo_documento, fecha_vencimiento_doc}, \ldots\}$$

Se identificó la dependencia funcional:

$$\text{nro_documento} \to \text{tipo_documento}$$

Dado que `nro_documento` no es superclave de `VALIDACION_KYC` (la clave primaria es `id_validación`), esta dependencia violaba BCNF.

Solución aplicada: Se extrajo la dependencia a una nueva tabla:

- `DOCUMENTO_IDENTIDAD(nro_documento PK, id_tipo_doc FK)`
- `VALIDACION_KYC` ahora referencia `nro_documento` como FK, eliminando la redundancia.

Resultado: Eliminación de anomalías de actualización: cambiar el tipo de un documento ahora se realiza en un solo lugar.

2.1.2. Jerarquía de TARJETA: Débito vs. Crédito

La entidad `TARJETA` presentaba atributos condicionales:

si tipo_tarjeta = 'Crédito' $\rightarrow$ requiere: limite_crédito, tasa_interés, fecha_corte

si tipo_tarjeta = 'Débito' $\rightarrow$ no aplica: limite_crédito, tasa_interés

Esto generaba dependencias funcionales parciales respecto al discriminador `tipo_tarjeta`, violando BCNF.

Solución aplicada: Especialización mediante herencia de clave primaria:

- `TARJETA(nro_tarjeta PK, tipo_tarjeta, ...atributos comunes ...)`
- `TARJETA_DEBITO(nro_tarjeta PK/FK, fondo_asociado)`
- `TARJETA_CREDITO(nro_tarjeta PK/FK, limite_credito, tasa_interes, fecha_corte)`

Resultado: Eliminación de valores NULL en atributos no aplicables, y garantía de integridad semántica: una tarjeta de débito no puede tener límite de crédito.

2.1.3. Jerarquía de CUENTA: Ahorro vs. Corriente

En la versión previa, la tabla `CUENTA` concentraba atributos condicionados por `tipo_cuenta`, específicamente `tasa_interes` aplicable a ahorro y `limite_diario` con semántica distinta según subtipo.

Solución aplicada: Especialización con superclase/subclases:

- `CUENTA(nro_cuenta PK, id_cliente, id_canal_apertura, tipo_cuenta, fecha_apertura, saldo, moneda, estado)`
- `CUENTA_AHORRO(nro_cuenta PK/FK, limite_diario, tasa_interes)`
- `CUENTA_CORRIENTE(nro_cuenta PK/FK, limite_diario)`

Resultado: Eliminación de nulos no aplicables en la superclase y mayor consistencia semántica por subtipo de cuenta, manteniendo trazabilidad mediante PK/FK compartida.

2.2. Cuarta Forma Normal (4FN) y Quinta Forma Normal (5FN)

Una relación está en 4FN si, para toda dependencia multivaluada no trivial $X \twoheadrightarrow Y$, X es una superclave. Está en 5FN si toda dependencia de join es implicada por las claves candidatas.

2.2.1. Extracción de atributos multivaluados

Se identificaron tres casos de dependencias multivaluadas independientes:

1. **EMAIL**: Un cliente puede tener múltiples correos, independientes de sus teléfonos o cuentas.

$$\text{id_cliente} \twoheadrightarrow \text{direccion_email}$$

2. **TELEFONO**: Un cliente puede tener múltiples teléfonos, independientes de sus emails.

$$\text{id_cliente} \twoheadrightarrow \text{numero}$$

3. **PERMISOS_TARJETA**: Los permisos (ecommerce, internacional) son independientes entre sí y de otros atributos de la tarjeta.

$$\text{nro_tarjeta} \twoheadrightarrow \{\text{permiso_ecommerce, permiso_internacional}\}$$

Solución aplicada: Extracción a tablas independientes con clave foránea:

- `EMAIL(direccion_email PK, id_cliente FK, ...)`
- `TELEFONO(numero PK, id_cliente FK, ...)`
- `PERMISOS_TARJETA(nro_tarjeta PK/FK, permiso_ecommerce, permiso_internacional)`

2.2.2. Demostración de reunión sin pérdida mediante algoritmo de chase

Para verificar que la descomposición de `CLIENTE` en `EMAIL` y `TELEFONO` no introduce tuplas espurias, se usa el algoritmo de *chase* sobre la relación auxiliar:

$$R(id_cliente, direccion_email, numero)$$

con la descomposición:

$$R_1(id_cliente, direccion_email), \qquad R_2(id_cliente, numero)$$

La matriz inicial del chase se construye así:

	$id_cliente$	$direccion_email$	$numero$
R_1	a_1	a_2	b_{13}
R_2	a_1	b_{22}	a_3

Como `id_cliente` es la clave común y conserva el mismo símbolo distinguido en ambas filas, el chase propaga esa igualdad y permite reconstruir la relación original sin pérdida de información. La matriz resultante queda:

	$id_cliente$	$direccion_email$	$numero$
R_1	a_1	a_2	a_3
R_2	a_1	a_2	a_3

Al obtener una fila completamente distinguida, se concluye que la descomposición es de **reunión sin pérdida**:

$$R = R_1 \bowtie R_2$$

sin generar tuplas espurias.

Resultado 4FN: Eliminación de tuplas espurias en joins.

Resultado 5FN: Garantía de que la reconstrucción mediante join natural no pierde información, ya que las dependencias de join están implicadas por las claves foráneas.

3. Justificación de interoperabilidad: tabla MOVIMIENTO

La tabla `MOVIMIENTO` declara los atributos `nro_cuenta_origen` y `nro_cuenta_destino` como `VARCHAR(20)` **sin restricción de llave foránea estricta** hacia la tabla `CUENTA` de este esquema.

Justificación: USB Bank debe registrar transferencias *interbancarias*, donde la cuenta de destino (o de origen) pertenece a una institución externa no modelada en esta base de datos. Imponer una FK estricta a `CUENTA` provocaría errores de integridad referencial al intentar registrar una transferencia hacia, por ejemplo, una cuenta del Banco Mercantil (id_banco = 5) cuyo número de cuenta no existe en `usb_bank.CUENTA`.

Mecanismo de compensación:

- Los atributos `id_banco_origen` y `id_banco_destino` (FK a tabla `BANCO`) permiten identificar la institución de cada cuenta.
- Si `id_banco_origen = id_banco_destino = 1` (USB Bank), entonces ambas cuentas *deben* existir en `CUENTA` (validación a nivel de aplicación).
- Si los bancos difieren, el sistema acepta el registro para garantizar trazabilidad del flujo de fondos, delegando la validación de contraparte al sistema de compensación interbancario.

Conclusión: Esta decisión de diseño prioriza la *disponibilidad y trazabilidad* sobre la integridad referencial estricta, conforme al teorema CAP, y es una práctica común en sistemas bancarios reales.

4. Restricción matemática 1:1: tabla AFILIACION_PAGOMOVIL

La funcionalidad de Pago Móvil requiere garantizar que **un número de teléfono esté afiliado a exactamente una cuenta** en USB Bank, para prevenir ambigüedades en la recepción de fondos.

Modelado aplicado:

```sql
CREATE TABLE "AFILIACION_PAGOMOVIL" (
    numero_telefono VARCHAR(20) PRIMARY KEY,
    nro_cuenta VARCHAR(20) NOT NULL,
    fecha_afiliacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_afilp2p_telefono
        FOREIGN KEY (numero_telefono) REFERENCES "TELEFONO"(numero) ON DELETE CASCADE,
    CONSTRAINT fk_afilp2p_cuenta
        FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE
);
```

Listing 1: Definición de la tabla de afiliación de Pago Móvil

Demostración de restricción 1:1:

- `numero_telefono` es **PRIMARY KEY** $\Rightarrow$ máximo 1 registro por teléfono.
- `nro_cuenta` es **NOT NULL** $\Rightarrow$ mínimo 1 cuenta por afiliación.
- La FK a `TELEFONO` garantiza que el teléfono pertenezca a un cliente válido.
- La FK a `CUENTA` garantiza que la cuenta exista y esté activa.

Resultado: Matemáticamente, la relación es **inyectiva y total** desde `numero_telefono` hacia `nro_cuenta`, cumpliendo la restricción de negocio: *1 teléfono* $\leftrightarrow$ *1 cuenta*.

5. Modelo conceptual y lógico

Esta sección documenta la vista previa al modelo relacional, integrando el diagrama entidad-relación extendido como puente entre el análisis conceptual y la implementación.

5.1. Diagrama entidad-relación extendido

El diagrama entidad-relación extendido sintetiza la vista conceptual del dominio, incluyendo las especializaciones de CLIENTE, CANAL, CUENTA y TARJETA, junto con las cardinalidades y restricciones que justifican el diseño relacional posterior.

5.2. Esquema entidad relación extendido

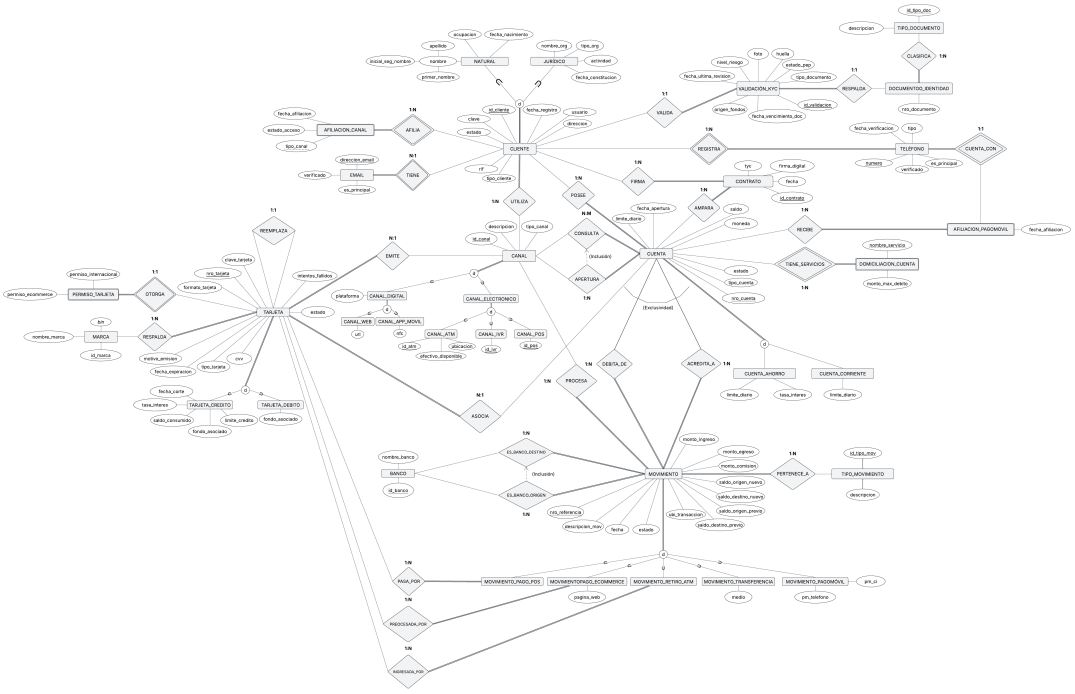

Figura 1: Esquema EER, Fase III

6. Modelo relacional

El modelo relacional define la estructura lógica de los datos organizada en **37 tablas**. A continuación, se presenta el esquema en notación moderna, representando gráficamente las tablas con sus atributos, tipos de datos, claves primarias y foráneas.

6.1. Esquema relacional base

El esquema relacional base corresponde a la traducción inicial del modelo EER a tablas, manteniendo la estructura conceptual antes de la depuración final por normalización. Esta vista permite documentar la transición entre el modelo conceptual y el modelo físico definitivo.

6.2. Esquema relacional base

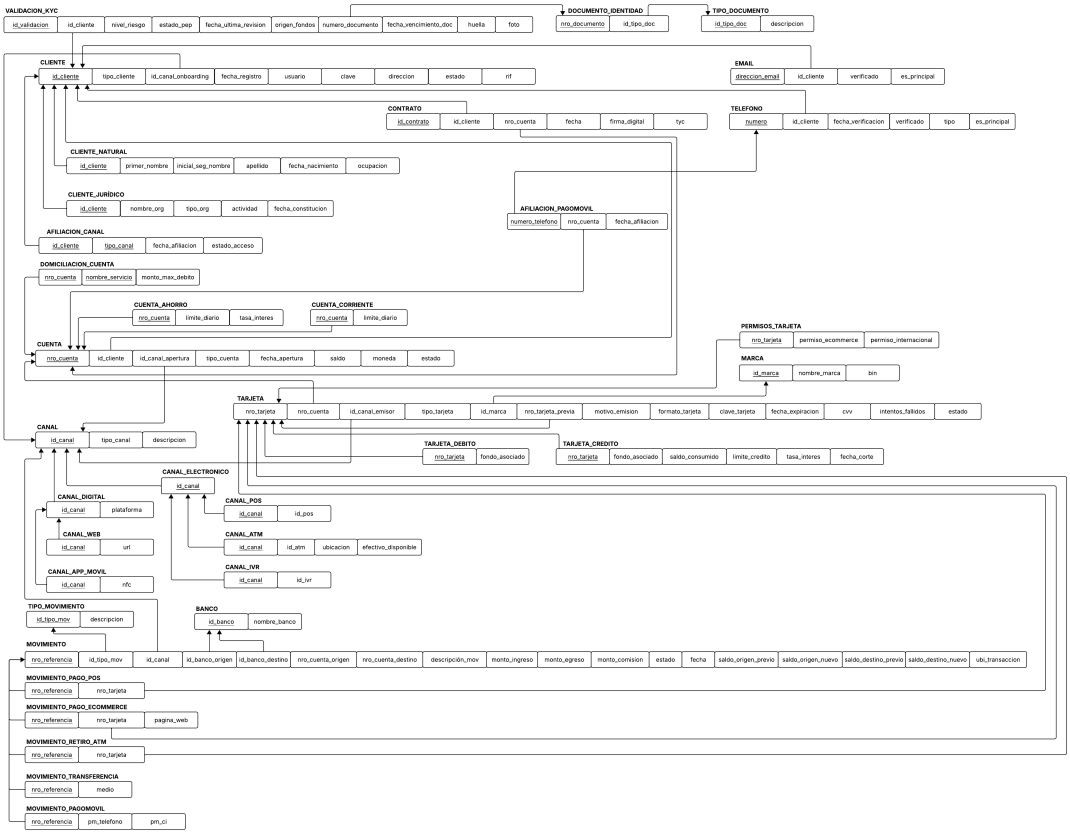

Figura 2: Esquema relacional base, Fase III

Módulos identificados:

- **Catálogos Base:** BANCO, MARCA, TIPO_MOVIMIENTO, TIPO_DOCUMENTO
- **Identidad:** DOCUMENTO_IDENTIDAD
- **Canales:** CANAL (padre), CANAL_DIGITAL, CANAL_WEB, CANAL_APP_MOVIL, CANAL_ELECTRONICO, CANAL_ATM, CANAL_POS, CANAL_IVR
- **Clientes/KYC:** CLIENTE (padre), CLIENTE_NATURAL, CLIENTE_JURIDICO, VALIDACION_KYC, EMAIL, TELEFONO, AFILIACION_CANAL
- **Productos:** CUENTA (padre), CUENTA_AHORRO, CUENTA_CORRIENTE, CONTRATO, DOMICILIACION_CUENTA, AFILIACION_PAGOMOVIL, TARJETA (padre), TARJETA_DEBITO, TARJETA_CREDITO, PERMISOS_TARJETA

- **Transacciones:** MOVIMIENTO (padre), MOVIMIENTO_PAGO_POS, MOVIMIENTO_PAGO_ECOMMERCE, MOVIMIENTO_RETIRO_ATM, MOVIMIENTO_TRANSFERENCIA, MOVIMIENTO_PAGOMOVIL
- **Auditoría:** REPORTE_CLIENTES_ELIMINADOS

6.3. Esquema relacional en notación moderna

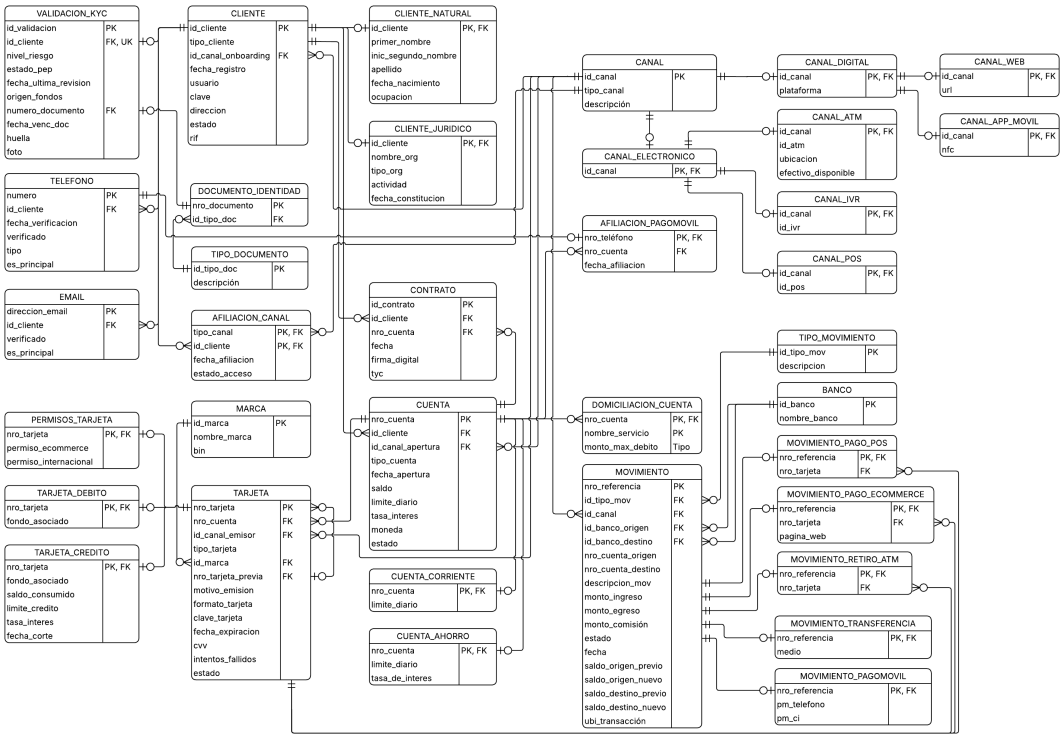

Figura 3: Esquema relacional en notación moderna (37 tablas, Fase III)

7. Diccionario de datos físico

El diccionario de datos detalla la definición técnica de cada atributo dentro de las 37 tablas del sistema. El esquema ha sido desplegado bajo el namespace `usb_bank`. Se emplea notación estricta en mayúsculas para las tablas.

Convenciones: PK (Clave Primaria), FK (Clave Foránea), NN (Not Null), UQ (Unique), CHK (Check Constraint).

7.1. Módulo 1: Catálogos e Identidad

7.1.1. 1. BANCO

Descripción: Catálogo de instituciones bancarias para operaciones interbancarias.

Atributo	Tipo	Restricciones
id_banco	SERIAL	PK
nombre_banco	VARCHAR(100)	NN, UQ

7.1.2. 2. MARCA

Descripción: Marcas de tarjetas (Visa, Mastercard, etc.).

Atributo	Tipo	Restricciones
id_marca	SERIAL	PK
nombre_marca	VARCHAR(50)	NN, UQ
bin	VARCHAR(6)	NULL

7.1.3. 3. TIPO_MOVIMIENTO

Descripción: Catálogo de tipos de transacciones.

Atributo	Tipo	Restricciones
id_tipo_mov	SERIAL	PK
descripcion	VARCHAR(100)	NN

7.1.4. 4. TIPO_DOCUMENTO

Descripción: Tipos de documentos de identidad.

Atributo	Tipo	Restricciones
id_tipo_doc	VARCHAR(10)	PK
descripcion	VARCHAR(100)	NN

7.1.5. 5. DOCUMENTO_IDENTIDAD (BCNF)

Descripción: Tabla extraída para cumplir BCNF: normaliza tipo de documento.

Atributo	Tipo	Restricciones
nro_documento	VARCHAR(20)	PK
id_tipo_doc	VARCHAR(10)	FK $\rightarrow$ TIPO_DOCUMENTO, NN

7.2. Módulo 2: Canales (Jerarquía de especialización)

7.2.1. 6. CANAL

Descripción: Superclase para todos los canales de acceso.

Atributo	Tipo	Restricciones
id_canal	SERIAL	PK
tipo_canal	VARCHAR(50)	NN, CHK $\in$ {'Digital','Electronico'}
descripcion	VARCHAR(100)	NN

7.2.2. 7-13. Subclases de CANAL

Las tablas `CANAL_DIGITAL`, `CANAL_WEB`, `CANAL_APP_MOVIL`, `CANAL_ELECTRONICO`, `CANAL_ATM`, `CANAL_POS`, `CANAL_IVR` implementan especialización mediante PK/FK compartida con `CANAL`, con atributos específicos según el subtipo.

7.3. Módulo 3: Clientes y KYC

7.3.1. 14. CLIENTE

Descripción: Superclase para clientes naturales y jurídicos.

Atributo	Tipo	Restricciones
id_cliente	SERIAL	PK
tipo_cliente	VARCHAR(20)	NN, CHK $\in$ {'Natural','Juridico'}
id_canal_onboarding	INTEGER	FK $\rightarrow$ CANAL, NN
fecha_registro	TIMESTAMP	NN, DEFAULT CURRENT_TIMESTAMP
usuario	VARCHAR(50)	NN, UQ
clave	VARCHAR(255)	NN (hash)
direccion	TEXT	NN
estado	VARCHAR(20)	NN, DEFAULT 'activo'
rif	VARCHAR(20)	NULL (solo jurídicos)

7.3.2. 15-16. CLIENTE_NATURAL / CLIENTE_JURIDICO

Especializaciones con PK/FK hacia CLIENTE, con atributos específicos según tipo de cliente.

7.3.3. 17. VALIDACION_KYC (BCNF)

Descripción: Expediente KYC normalizado (referencia a DOCUMENTO_IDENTIDAD).

Atributo	Tipo	Restricciones
id_validacion	SERIAL	PK
id_cliente	INTEGER	FK $\rightarrow$ CLIENTE, UQ, NN
nivel_riesgo	VARCHAR(20)	NN
estado_pep	BOOLEAN	NN, DEFAULT false
fecha_ultima_revision	DATE	NN
origen_fondos	VARCHAR(100)	NN
numero_documento	VARCHAR(20)	FK $\rightarrow$ DOCUMENTO_IDENTIDAD, UQ, NN
fecha_vencimiento_doc	DATE	NN
huella, foto	VARCHAR(255)	NULL (rutas/hashes)

7.3.4. 18-19. EMAIL / TELEFONO (4FN)

Tablas extraídas para cumplir 4FN: atributos multivaluados independientes.

7.3.5. 20. AFILIACION_CANAL

Descripción: Canales autorizados por cliente (relación N:M resuelta).

7.4. Módulo 4: Productos Financieros

7.4.1. 21. CUENTA

Descripción: Cuentas bancarias (Ahorro/Corriente).

Atributo	Tipo	Restricciones
nro_cuenta	VARCHAR(20)	PK
id_cliente	INTEGER	FK $\rightarrow$ CLIENTE, NN
id_canal_apertura	INTEGER	FK $\rightarrow$ CANAL, NN
tipo_cuenta	VARCHAR(20)	NN, CHK $\in$ {'Ahorro','Corriente'}
fecha_apertura	TIMESTAMP	NN, DEFAULT CURRENT_TIMESTAMP
saldo	NUMERIC(15,2)	NN, DEFAULT 0.00
moneda	VARCHAR(3)	NN, DEFAULT 'VES'
estado	VARCHAR(20)	NN, DEFAULT 'activa'

7.4.2. 22-23. CUENTA_AHORRO / CUENTA_CORRIENTE (BCNF)

Especializaciones con PK/FK compartida hacia `CUENTA`; `CUENTA_AHORRO` define `limite_diario` y `tasa_interes`, mientras `CUENTA_CORRIENTE` define `limite_diario`.

7.4.3. 24-26. CONTRATO / DOMICILIACION_CUENTA / AFILIACION_PAGOMOVIL

Tablas auxiliares para contratos, domiciliaciones y la restricción 1:1 de Pago Móvil.

7.4.4. 27. TARJETA (BCNF)

Descripción: Superclase para tarjetas de débito y crédito.

Atributo	Tipo	Restricciones
nro_tarjeta	VARCHAR(16)	PK
nro_cuenta	VARCHAR(20)	FK $\rightarrow$ CUENTA, NN
id_canal_emisor	INTEGER	FK $\rightarrow$ CANAL, NN
tipo_tarjeta	VARCHAR(20)	NN, CHK $\in$ {'Debito','Credito'}
id_marca	INTEGER	FK $\rightarrow$ MARCA, NN
formato_tarjeta	VARCHAR(20)	NN
clave_tarjeta	VARCHAR(255)	NN (hash)
fecha_expiracion	DATE	NN
cvv	VARCHAR(4)	NN
estado	VARCHAR(20)	NN, DEFAULT 'activa'

7.4.5. 28-29. TARJETA_DEBITO / TARJETA_CREDITO (BCNF)

Especializaciones con atributos específicos; evitan NULLs en campos no aplicables.

7.4.6. 30. PERMISOS_TARJETA (4FN)

Descripción: Permisos independientes (ecommerce, internacional) extraídos para cumplir 4FN.

7.5. Módulo 5: Transacciones y Auditoría

7.5.1. 31. MOVIMIENTO

Descripción: Núcleo contable. Registra todos los flujos de dinero.

Los campos `nro_cuenta_origen/destino` se definen como `VARCHAR` sin FK estricta a `CUENTA` para permitir transferencias interbancarias (ver Sección 3).

Atributo	Tipo	Restricciones
nro_referencia	VARCHAR(50)	PK
id_tipo_mov	INTEGER	FK → TIPO_MOVIMIENTO, NN
id_canal	INTEGER	FK → CANAL, NN
id_banco_origen/destino	INTEGER	FK → BANCO, NN
nro_cuenta_origen/destino	VARCHAR(20)	NN
descripcion_mov	VARCHAR(255)	NN
monto_ingreso/egreso	NUMERIC(15,2)	NN, DEFAULT 0.00
monto_comision	NUMERIC(15,2)	NN, DEFAULT 0.00
estado	VARCHAR(20)	NN
fecha	TIMESTAMP	NN, DEFAULT CURRENT_TIMESTAMP
saldo_origen_previo/nuevo	NUMERIC(15,2)	NN
saldo_destino_previo/nuevo	NUMERIC(15,2)	NULL
ubi_transaccion	VARCHAR(255)	NULL
CHK: `nro_cuenta_origen <>nro_cuenta_destino`		

7.5.2. 32-36. Subclases de MOVIMIENTO

Las subclases de `MOVIMIENTO` son:

- `MOVIMIENTO_PAGO_POS`
- `MOVIMIENTO_PAGO_ECOMMERCE`
- `MOVIMIENTO_RETIRO_ATM`
- `MOVIMIENTO_TRANSFERENCIA`
- `MOVIMIENTO_PAGOMOVIL`

Estas especializaciones contienen atributos específicos del tipo de transacción.

7.5.3. 37. REPORTE_CLIENTES_ELIMINADOS (Auditoría)

Descripción: Tabla de auditoría poblada por trigger ante eliminación de clientes inactivos.

8. Restricciones de integridad

El diseño incorpora restricciones de integridad para garantizar consistencia:

8.1. Integridad de entidad

- Todas las tablas tienen PRIMARY KEY que identifica unívocamente cada registro.
- Claves primarias no permiten NULL.

8.2. Integridad referencial

- FOREIGN KEY mantienen consistencia entre tablas relacionadas.
- Especializaciones usan PK compartida (herencia).
- **Excepción justificada:** los campos `MOVIMIENTO.nro_cuenta_origen` y `MOVIMIENTO.nro_cuenta_destino` se manejan sin FK estricta (ver Sección 3).

8.3. Restricciones de dominio y negocio

- CHECK constraints validan valores permitidos (ej. tipos de cliente, tarjeta).
- NUMERIC(15,2) para montos monetarios.
- TIMESTAMP DEFAULT CURRENT_TIMESTAMP para auditoría.
- Trigger `auditar_eliminacion_cliente()` para trazabilidad de bajas.

9. Anexos: Código fuente implementado

9.1. A. Script maestro SQL: creación de tablas completa

```sql
--- CONFIGURACION DE ESQUEMA NORMALIZADO ---
-- Representa el estado de la BD después de aplicar BCNF, 4FN y 5FN
DROP SCHEMA IF EXISTS "usb_bank" CASCADE;
CREATE SCHEMA "usb_bank";
SET search_path TO "usb_bank";

--- CATALOGOS BASE ---
CREATE TABLE "BANCO" (
    id_banco SERIAL PRIMARY KEY,
    nombre_banco VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE "MARCA" (
    id_marca SERIAL PRIMARY KEY,
```

```sql
    nombre_marca VARCHAR(50) NOT NULL UNIQUE,
    bin VARCHAR(6)
);

CREATE TABLE "TIPO_MOVIMIENTO" (
    id_tipo_mov SERIAL PRIMARY KEY,
    descripcion VARCHAR(100) NOT NULL
);

CREATE TABLE "TIPO_DOCUMENTO" (
    id_tipo_doc VARCHAR(10) PRIMARY KEY,
    descripcion VARCHAR(100) NOT NULL
);

--- ========================================================== ---
--- MODIFICACIÓN BCNF (1.1): EXTRACCIÓN DE IDENTIDAD
--- ========================================================== ---
CREATE TABLE "DOCUMENTO_IDENTIDAD" (
    nro_documento VARCHAR(20) PRIMARY KEY,
    id_tipo_doc VARCHAR(10) NOT NULL,
    CONSTRAINT fk_doc_tipo FOREIGN KEY (id_tipo_doc) REFERENCES "TIPO_DOCUMENTO"(id_tipo_doc)
);

--- CANALES ---
CREATE TABLE "CANAL" (
    id_canal SERIAL PRIMARY KEY,
    tipo_canal VARCHAR(50) NOT NULL,
    descripcion VARCHAR(100) NOT NULL,
    CONSTRAINT chk_tipo_canal CHECK (tipo_canal IN ('Digital', 'Electronico'))
);

CREATE TABLE "CANAL_DIGITAL" (
    id_canal INTEGER PRIMARY KEY,
    plataforma VARCHAR(50) NOT NULL,
    CONSTRAINT fk_cdigital_canal FOREIGN KEY (id_canal) REFERENCES "CANAL"(id_canal) ON DELETE CASCADE
);

CREATE TABLE "CANAL_WEB" (
    id_canal INTEGER PRIMARY KEY,
    url VARCHAR(255) NOT NULL,
    CONSTRAINT fk_cweb_cdigital FOREIGN KEY (id_canal) REFERENCES "CANAL_DIGITAL"(id_canal) ON DELETE CASCADE
);

CREATE TABLE "CANAL_APP_MOVIL" (
    id_canal INTEGER PRIMARY KEY,
    nfc BOOLEAN NOT NULL,
    CONSTRAINT fk_capp_cdigital FOREIGN KEY (id_canal) REFERENCES "CANAL_DIGITAL"(id_canal) ON DELETE CASCADE
);

CREATE TABLE "CANAL_ELECTRONICO" (
    id_canal INTEGER PRIMARY KEY,
    CONSTRAINT fk_celectronico_canal FOREIGN KEY (id_canal) REFERENCES "CANAL"(id_canal) ON DELETE CASCADE
);

CREATE TABLE "CANAL_ATM" (
    id_canal INTEGER PRIMARY KEY,
    id_atm VARCHAR(50) NOT NULL UNIQUE,
    ubicacion TEXT NOT NULL,
    efectivo_disponible NUMERIC(15,2) NOT NULL DEFAULT 0.00,
```

```sql
    CONSTRAINT fk_catm_celectronico FOREIGN KEY (id_canal) REFERENCES "CANAL_ELECTRONICO" (id_canal) ON DELETE CASCADE
);

CREATE TABLE "CANAL_POS" (
    id_canal INTEGER PRIMARY KEY,
    id_pos VARCHAR(50) NOT NULL UNIQUE,
    CONSTRAINT fk_cpos_celectronico FOREIGN KEY (id_canal) REFERENCES "CANAL_ELECTRONICO" (id_canal) ON DELETE CASCADE
);

CREATE TABLE "CANAL_IVR" (
    id_canal INTEGER PRIMARY KEY,
    id_ivr VARCHAR(50) NOT NULL UNIQUE,
    CONSTRAINT fk_civr_celectronico FOREIGN KEY (id_canal) REFERENCES "CANAL_ELECTRONICO" (id_canal) ON DELETE CASCADE
);

--- CLIENTES Y KYC ---
CREATE TABLE "CLIENTE" (
    id_cliente SERIAL PRIMARY KEY,
    tipo_cliente VARCHAR(20) NOT NULL,
    id_canal_onboarding INTEGER NOT NULL,
    fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    usuario VARCHAR(50) NOT NULL UNIQUE,
    clave VARCHAR(255) NOT NULL,
    direccion TEXT NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'activo',
    rif VARCHAR(20),
    CONSTRAINT chk_tipo_cliente CHECK (tipo_cliente IN ('Natural', 'Juridico')),
    CONSTRAINT fk_cliente_canal FOREIGN KEY (id_canal_onboarding) REFERENCES "CANAL"(id_canal)
);

CREATE TABLE "CLIENTE_NATURAL" (
    id_cliente INTEGER PRIMARY KEY,
    primer_nombre VARCHAR(50) NOT NULL,
    inicial_seg_nombre VARCHAR(50),
    apellido VARCHAR(50) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    ocupacion VARCHAR(100),
    CONSTRAINT fk_cnatural_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE
);

CREATE TABLE "CLIENTE_JURIDICO" (
    id_cliente INTEGER PRIMARY KEY,
    nombre_org VARCHAR(100) NOT NULL,
    tipo_org VARCHAR(50) NOT NULL,
    actividad VARCHAR(100) NOT NULL,
    fecha_constitucion DATE NOT NULL,
    CONSTRAINT chk_tipo_org CHECK (tipo_org IN ('C.A.', 'S.A.', 'S.R.L.', 'Firma Personal', 'Cooperativa', 'Gubernamental')),
    CONSTRAINT fk_cjuridico_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE
);

--- MODIFICACIÓN BCNF (1.1): VALIDACION KYC LIMPIA ---
-- Se eliminó tipo_documento. Ahora depende directamente de DOCUMENTO_IDENTIDAD
CREATE TABLE "VALIDACION_KYC" (
    id_validacion SERIAL PRIMARY KEY,
    id_cliente INTEGER NOT NULL UNIQUE,
    nivel_riesgo VARCHAR(20) NOT NULL,
    estado_pep BOOLEAN NOT NULL DEFAULT false,
    fecha_ultima_revision DATE NOT NULL,
```

```sql
    origen_fondos VARCHAR(100) NOT NULL,
    numero_documento VARCHAR(20) NOT NULL UNIQUE,
    fecha_vencimiento_doc DATE NOT NULL,
    huella VARCHAR(255),
    foto VARCHAR(255),
    CONSTRAINT fk_kyc_documento FOREIGN KEY (numero_documento) REFERENCES "DOCUMENTO_IDENTIDAD"(nro_documento),
    CONSTRAINT fk_kyc_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE
);

CREATE TABLE "EMAIL" (
    direccion_email VARCHAR(100) PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    verificado BOOLEAN NOT NULL DEFAULT false,
    es_principal BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT chk_formato_email CHECK (direccion_email ~ '^[A-Za-z0-9._%-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$'),
    CONSTRAINT fk_email_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE
);

CREATE TABLE "TELEFONO" (
    numero VARCHAR(20) PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    fecha_verificacion TIMESTAMP,
    verificado BOOLEAN NOT NULL DEFAULT false,
    tipo VARCHAR(20) NOT NULL,
    es_principal BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT chk_tipo_telefono CHECK (tipo IN ('Movil', 'Fijo')),
    CONSTRAINT fk_telefono_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE
);

CREATE TABLE "AFILIACION_CANAL" (
    id_cliente INTEGER NOT NULL,
    tipo_canal VARCHAR(50) NOT NULL,
    fecha_afiliacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado_acceso VARCHAR(20) NOT NULL DEFAULT 'activo',
    PRIMARY KEY (id_cliente, tipo_canal),
    CONSTRAINT chk_tipo_canal_afil CHECK (tipo_canal IN ('Digital', 'Electronico')),
    CONSTRAINT fk_afil_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE
);

--- PRODUCTOS FINANCIEROS ---
CREATE TABLE "CUENTA" (
    nro_cuenta VARCHAR(20) PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    id_canal_apertura INTEGER NOT NULL,
    tipo_cuenta VARCHAR(20) NOT NULL,
    fecha_apertura TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    saldo NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    moneda VARCHAR(3) NOT NULL DEFAULT 'VES',
    estado VARCHAR(20) NOT NULL DEFAULT 'activa',
    CONSTRAINT chk_tipo_cuenta CHECK (tipo_cuenta IN ('Ahorro', 'Corriente')),
    CONSTRAINT fk_cuenta_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente),
    CONSTRAINT fk_cuenta_canal FOREIGN KEY (id_canal_apertura) REFERENCES "CANAL"(id_canal)
);

CREATE TABLE "CUENTA_AHORRO" (
    nro_cuenta VARCHAR(20) PRIMARY KEY,
    limite_diario NUMERIC(15,2) NOT NULL,
```

```sql
    tasa_interes NUMERIC(5,2) NOT NULL,
    CONSTRAINT chk_tasa_ahorro CHECK (tasa_interes >= 0),
    CONSTRAINT fk_cahorro_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE
);

CREATE TABLE "CUENTA_CORRIENTE" (
    nro_cuenta VARCHAR(20) PRIMARY KEY,
    limite_diario NUMERIC(15,2) NOT NULL,
    CONSTRAINT fk_ccorriente_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE
);

CREATE TABLE "CONTRATO" (
    id_contrato SERIAL PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    nro_cuenta VARCHAR(20) NOT NULL,
    fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    firma_digital VARCHAR(255) NOT NULL,
    tyc TEXT NOT NULL,
    CONSTRAINT fk_contrato_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE,
    CONSTRAINT fk_contrato_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE
);

CREATE TABLE "DOMICILIACION_CUENTA" (
    nro_cuenta VARCHAR(20),
    nombre_servicio VARCHAR(100),
    monto_max_debito NUMERIC(15,2) NOT NULL,
    PRIMARY KEY (nro_cuenta, nombre_servicio),
    CONSTRAINT fk_domic_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE
);

CREATE TABLE "AFILIACION_PAGOMOVIL" (
    numero_telefono VARCHAR(20) PRIMARY KEY,
    nro_cuenta VARCHAR(20) NOT NULL,
    fecha_afiliacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_afilp2p_telefono FOREIGN KEY (numero_telefono) REFERENCES "TELEFONO"(numero) ON DELETE CASCADE,
    CONSTRAINT fk_afilp2p_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE
);

--- ========================================================== ---
--- MODIFICACIÓN BCNF y 4FN/5FN: JERARQUÍA DE TARJETAS
--- ========================================================== ---

-- Tabla Padre TARJETA limpia (Se removieron límites, saldos y permisos)
CREATE TABLE "TARJETA" (
    nro_tarjeta VARCHAR(16) PRIMARY KEY,
    nro_cuenta VARCHAR(20) NOT NULL,
    id_canal_emisor INTEGER NOT NULL,
    tipo_tarjeta VARCHAR(20) NOT NULL,
    id_marca INTEGER NOT NULL,
    nro_tarjeta_previa VARCHAR(16),
    motivo_emision VARCHAR(50),
    formato_tarjeta VARCHAR(20) NOT NULL,
    clave_tarjeta VARCHAR(255) NOT NULL,
    fecha_expiracion DATE NOT NULL,
    cvv VARCHAR(4) NOT NULL,
    intentos_fallidos INTEGER NOT NULL DEFAULT 0,
    estado VARCHAR(20) NOT NULL DEFAULT 'activa',
    CONSTRAINT chk_tipo_tarjeta CHECK (tipo_tarjeta IN ('Debito', 'Credito')),
```

```sql
    CONSTRAINT fk_tarjeta_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta),
    CONSTRAINT fk_tarjeta_canal FOREIGN KEY (id_canal_emisor) REFERENCES "CANAL"(id_canal),
    CONSTRAINT fk_tarjeta_marca FOREIGN KEY (id_marca) REFERENCES "MARCA"(id_marca),
    CONSTRAINT fk_tarjeta_previa FOREIGN KEY (nro_tarjeta_previa) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE SET NULL
);

-- Subclase Débito (1.1 BCNF)
CREATE TABLE "TARJETA_DEBITO" (
    nro_tarjeta VARCHAR(16) PRIMARY KEY,
    fondo_asociado VARCHAR(50) NOT NULL,
    CONSTRAINT fk_tdebito_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE CASCADE
);

-- Subclase Crédito (1.1 BCNF)
CREATE TABLE "TARJETA_CREDITO" (
    nro_tarjeta VARCHAR(16) PRIMARY KEY,
    fondo_asociado VARCHAR(50) NOT NULL,
    saldo_consumido NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    limite_credito NUMERIC(15,2) NOT NULL,
    tasa_interes NUMERIC(5,2) NOT NULL,
    fecha_corte DATE NOT NULL,
    CONSTRAINT fk_tcredito_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE CASCADE
);

-- Extracción de Dependencias Multivaluadas (1.2 4FN/5FN)
CREATE TABLE "PERMISOS_TARJETA" (
    nro_tarjeta VARCHAR(16) PRIMARY KEY,
    permiso_ecommerce BOOLEAN NOT NULL DEFAULT false,
    permiso_internacional BOOLEAN NOT NULL DEFAULT false,
    CONSTRAINT fk_permisos_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE CASCADE
);

--- TRANSACCIONES ---
CREATE TABLE "MOVIMIENTO" (
    nro_referencia VARCHAR(50) PRIMARY KEY,
    id_tipo_mov INTEGER NOT NULL,
    id_canal INTEGER NOT NULL,
    id_banco_origen INTEGER NOT NULL,
    id_banco_destino INTEGER NOT NULL,
    nro_cuenta_origen VARCHAR(20) NOT NULL,
    nro_cuenta_destino VARCHAR(20) NOT NULL,
    descripcion_mov VARCHAR(255) NOT NULL,
    monto_ingreso NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    monto_egreso NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    monto_comision NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    estado VARCHAR(20) NOT NULL,
    fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    saldo_origen_previo NUMERIC(15,2) NOT NULL,
    saldo_origen_nuevo NUMERIC(15,2) NOT NULL,
    saldo_destino_previo NUMERIC(15,2),
    saldo_destino_nuevo NUMERIC(15,2),
    ubi_transaccion VARCHAR(255),

    CONSTRAINT fk_mov_tipo FOREIGN KEY (id_tipo_mov) REFERENCES "TIPO_MOVIMIENTO"(id_tipo_mov),
    CONSTRAINT fk_mov_canal FOREIGN KEY (id_canal) REFERENCES "CANAL"(id_canal),
    CONSTRAINT fk_mov_banco_origen FOREIGN KEY (id_banco_origen) REFERENCES "BANCO"(id_banco),
```

```sql
    CONSTRAINT fk_mov_banco_destino FOREIGN KEY (id_banco_destino) REFERENCES "BANCO"(id_banco),
    CONSTRAINT chk_mov_cuentas_distintas CHECK (nro_cuenta_origen <> nro_cuenta_destino)
);

CREATE TABLE "MOVIMIENTO_PAGO_POS" (
    nro_referencia VARCHAR(50) PRIMARY KEY,
    nro_tarjeta VARCHAR(16) NOT NULL,
    CONSTRAINT fk_mpos_movimiento FOREIGN KEY (nro_referencia) REFERENCES "MOVIMIENTO"(nro_referencia) ON DELETE CASCADE,
    CONSTRAINT fk_mpos_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta)
);

CREATE TABLE "MOVIMIENTO_PAGO_ECOMMERCE" (
    nro_referencia VARCHAR(50) PRIMARY KEY,
    nro_tarjeta VARCHAR(16) NOT NULL,
    pagina_web VARCHAR(255) NOT NULL,
    CONSTRAINT fk_mecom_movimiento FOREIGN KEY (nro_referencia) REFERENCES "MOVIMIENTO"(nro_referencia) ON DELETE CASCADE,
    CONSTRAINT fk_mecom_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta)
);

CREATE TABLE "MOVIMIENTO_RETIRO_ATM" (
    nro_referencia VARCHAR(50) PRIMARY KEY,
    nro_tarjeta VARCHAR(16) NOT NULL,
    CONSTRAINT fk_matm_movimiento FOREIGN KEY (nro_referencia) REFERENCES "MOVIMIENTO"(nro_referencia) ON DELETE CASCADE,
    CONSTRAINT fk_matm_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta)
);

CREATE TABLE "MOVIMIENTO_TRANSFERENCIA" (
    nro_referencia VARCHAR(50) PRIMARY KEY,
    medio VARCHAR(50) NOT NULL,
    CONSTRAINT fk_mtransf_movimiento FOREIGN KEY (nro_referencia) REFERENCES "MOVIMIENTO" (nro_referencia) ON DELETE CASCADE
);

CREATE TABLE "MOVIMIENTO_PAGOMOVIL" (
    nro_referencia VARCHAR(50) PRIMARY KEY,
    pm_telefono VARCHAR(20) NOT NULL,
    pm_ci VARCHAR(20) NOT NULL,
    CONSTRAINT fk_mpagomovil_movimiento FOREIGN KEY (nro_referencia) REFERENCES "MOVIMIENTO"(nro_referencia) ON DELETE CASCADE
);

-- ==============================================================================
-- MÓDULO DE AUDITORÍA Y LIMPIEZA (Adaptado a Esquema usb_bank BCNF/5FN)
-- ==============================================================================
SET search_path TO "usb_bank";

-- 1. CORRECCIÓN DE BORRADOS EN CASCADA
ALTER TABLE "CUENTA" DROP CONSTRAINT IF EXISTS fk_cuenta_cliente;
ALTER TABLE "CUENTA" ADD CONSTRAINT fk_cuenta_cliente FOREIGN KEY (id_cliente) REFERENCES "CLIENTE"(id_cliente) ON DELETE CASCADE;

ALTER TABLE "TARJETA" DROP CONSTRAINT IF EXISTS fk_tarjeta_cuenta;
ALTER TABLE "TARJETA" ADD CONSTRAINT fk_tarjeta_cuenta FOREIGN KEY (nro_cuenta) REFERENCES "CUENTA"(nro_cuenta) ON DELETE CASCADE;

ALTER TABLE "MOVIMIENTO_PAGO_POS" DROP CONSTRAINT IF EXISTS fk_mpos_tarjeta;
ALTER TABLE "MOVIMIENTO_PAGO_POS" ADD CONSTRAINT fk_mpos_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE CASCADE;
```

```sql

ALTER TABLE "MOVIMIENTO_PAGO_ECOMMERCE" DROP CONSTRAINT IF EXISTS fk_mecom_tarjeta;
ALTER TABLE "MOVIMIENTO_PAGO_ECOMMERCE" ADD CONSTRAINT fk_mecom_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE CASCADE;

ALTER TABLE "MOVIMIENTO_RETIRO_ATM" DROP CONSTRAINT IF EXISTS fk_matm_tarjeta;
ALTER TABLE "MOVIMIENTO_RETIRO_ATM" ADD CONSTRAINT fk_matm_tarjeta FOREIGN KEY (nro_tarjeta) REFERENCES "TARJETA"(nro_tarjeta) ON DELETE CASCADE;

-- 2. TABLA DE AUDITORÍA
CREATE TABLE IF NOT EXISTS "REPORTE_CLIENTES_ELIMINADOS" (
    id_reporte SERIAL PRIMARY KEY,
    fecha_eliminacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_cliente INTEGER NOT NULL,
    doc_cliente VARCHAR(30),
    nombre_cliente VARCHAR(150) NOT NULL,
    cantidad_cuentas INTEGER NOT NULL DEFAULT 0
);

-- 3. FUNCIÓN Y TRIGGER
CREATE OR REPLACE FUNCTION auditar_eliminacion_cliente()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_documento VARCHAR(30); v_nombre VARCHAR(150); v_cant_cuentas INTEGER;
BEGIN
    SELECT COALESCE(k.numero_documento, OLD.rif, 'SIN DOC') INTO v_documento FROM "VALIDACION_KYC" k WHERE k.id_cliente = OLD.id_cliente;

    IF OLD.tipo_cliente = 'Natural' THEN
        SELECT COALESCE(cn.primer_nombre || ' ' || cn.apellido, 'SIN NOMBRE') INTO v_nombre FROM "CLIENTE_NATURAL" cn WHERE cn.id_cliente = OLD.id_cliente;
    ELSIF OLD.tipo_cliente = 'Juridico' THEN
        SELECT COALESCE(cj.nombre_org, 'SIN NOMBRE') INTO v_nombre FROM "CLIENTE_JURIDICO" cj WHERE cj.id_cliente = OLD.id_cliente;
    ELSE
        v_nombre := 'NO DEFINIDO';
    END IF;

    SELECT COUNT(*) INTO v_cant_cuentas FROM "CUENTA" WHERE id_cliente = OLD.id_cliente;
    INSERT INTO "REPORTE_CLIENTES_ELIMINADOS" (id_cliente, doc_cliente, nombre_cliente, cantidad_cuentas) VALUES (OLD.id_cliente, v_documento, v_nombre, v_cant_cuentas);
    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS tr_auditar_eliminacion ON "CLIENTE";
CREATE TRIGGER tr_auditar_eliminacion BEFORE DELETE ON "CLIENTE" FOR EACH ROW EXECUTE FUNCTION auditar_eliminacion_cliente();

-- 4. STORED PROCEDURE Y CURSOR (Para la defensa técnica)
CREATE OR REPLACE PROCEDURE "sp_mostrar_clientes_eliminados"()
LANGUAGE plpgsql AS $$
DECLARE
    v_id_cliente INTEGER; v_doc_cliente VARCHAR(30); v_nombre_cliente VARCHAR(150); v_cantidad_cuentas INTEGER;
    v_total_ingresos NUMERIC(15,2) := 0; v_total_egresos NUMERIC(15,2) := 0; v_saldo_total_banco NUMERIC(15,2) := 0; v_total_clientes INTEGER := 0;
    cur_clientes CURSOR FOR SELECT id_cliente, doc_cliente, nombre_cliente, cantidad_cuentas FROM "REPORTE_CLIENTES_ELIMINADOS" ORDER BY fecha_eliminacion DESC, id_cliente;
BEGIN
    RAISE NOTICE '=== REPORTE DE CLIENTES ELIMINADOS POR INACTIVIDAD ===';
    OPEN cur_clientes;
    LOOP
```

```
        FETCH cur_clientes INTO v_id_cliente, v_doc_cliente, v_nombre_cliente, v_cantidad_cuentas;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'ID: % | DOC: % | NOMBRE: % | CUENTAS: %', LPAD(v_id_cliente::text, 4, '0'), RPAD(COALESCE(v_doc_cliente, 'N/A'), 12, ' '), RPAD(v_nombre_cliente, 25, ' '), v_cantidad_cuentas;
    END LOOP;
    CLOSE cur_clientes;

    SELECT COUNT(DISTINCT id_cliente) INTO v_total_clientes FROM "REPORTE_CLIENTES_ELIMINADOS";
    SELECT COALESCE(SUM(monto_ingreso), 0) INTO v_total_ingresos FROM "MOVIMIENTO";
    SELECT COALESCE(SUM(monto_egreso), 0) INTO v_total_egresos FROM "MOVIMIENTO";
    SELECT COALESCE(SUM(saldo), 0) INTO v_saldo_total_banco FROM "CUENTA";

    RAISE NOTICE '=== RESUMEN GENERAL DEL BANCO (POST-ELIMINACIÓN) ===';
    RAISE NOTICE 'TOTAL CLIENTES ELIMINADOS: %', v_total_clientes;
    RAISE NOTICE 'TOTAL INGRESOS REGISTRADOS: % VES', v_total_ingresos;
    RAISE NOTICE 'TOTAL EGRESOS REGISTRADOS:  % VES', v_total_egresos;
    RAISE NOTICE 'SALDO TOTAL EN BÓVEDAS:     % VES', v_saldo_total_banco;
END;
$$;
```

Listing 2: Script 01 completo de creación de tablas del esquema usb_bank

9.2. B. Script de inserts: extracto representativo

```
SET search_path TO "usb_bank";

INSERT INTO "BANCO" (id_banco, nombre_banco) VALUES
(1, 'USB Bank'), (2, 'Banco Nacional del Caribe'), (3, 'Banco Capital Uno')
ON CONFLICT DO NOTHING;

INSERT INTO "CLIENTE" (
    id_cliente, tipo_cliente, id_canal_onboarding, fecha_registro, usuario, clave, direccion, estado, rif
) VALUES
(1, 'Natural', 4, '2020-01-15 10:00:00', 'mruiz01', 'hash_clave_001', 'Caracas, Chacao, Calle Sucre', 'activo', NULL),
(13, 'Juridico', 2, '2020-08-08 08:00:00', 'alfa_consulting', 'hash_clave_013', 'Caracas, Torre Financiera Alfa', 'activo', 'J-40100013-1')
ON CONFLICT DO NOTHING;

INSERT INTO "CUENTA" (
    nro_cuenta, id_cliente, id_canal_apertura, tipo_cuenta, fecha_apertura,
    saldo, moneda, estado
) VALUES
('01010000000000000001', 1, 1, 'Ahorro', '2020-01-16 10:00:00', 1500.00, 'VES', 'activa'),
('01010000000000000013', 13, 2, 'Corriente', '2020-08-09 08:00:00', 25000.00, 'VES', 'activa')
ON CONFLICT DO NOTHING;

INSERT INTO "CUENTA_AHORRO" (nro_cuenta, limite_diario, tasa_interes) VALUES
('01010000000000000001', 500.00, 2.5)
ON CONFLICT DO NOTHING;

INSERT INTO "CUENTA_CORRIENTE" (nro_cuenta, limite_diario) VALUES
('01010000000000000013', 5000.00)
ON CONFLICT DO NOTHING;
```

Listing 3: Extracto del script de carga inicial de datos (02_inserts_datos.sql)

9.3. C. Trigger de auditoría (SQL)

```sql
CREATE TABLE IF NOT EXISTS "REPORTE_CLIENTES_ELIMINADOS" (
    id_reporte SERIAL PRIMARY KEY,
    fecha_eliminacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_cliente INTEGER NOT NULL,
    doc_cliente VARCHAR(30),
    nombre_cliente VARCHAR(150) NOT NULL,
    cantidad_cuentas INTEGER NOT NULL DEFAULT 0
);

CREATE OR REPLACE FUNCTION auditar_eliminacion_cliente()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_documento VARCHAR(30); v_nombre VARCHAR(150); v_cant_cuentas INTEGER;
BEGIN
    SELECT COALESCE(k.numero_documento, OLD.rif, 'SIN DOC') INTO v_documento FROM "VALIDACION_KYC" k WHERE k.id_cliente = OLD.id_cliente;

    IF OLD.tipo_cliente = 'Natural' THEN
        SELECT COALESCE(cn.primer_nombre || ' ' || cn.apellido, 'SIN NOMBRE') INTO v_nombre FROM "CLIENTE_NATURAL" cn WHERE cn.id_cliente = OLD.id_cliente;
    ELSIF OLD.tipo_cliente = 'Juridico' THEN
        SELECT COALESCE(cj.nombre_org, 'SIN NOMBRE') INTO v_nombre FROM "CLIENTE_JURIDICO" cj WHERE cj.id_cliente = OLD.id_cliente;
    ELSE
        v_nombre := 'NO DEFINIDO';
    END IF;

    SELECT COUNT(*) INTO v_cant_cuentas FROM "CUENTA" WHERE id_cliente = OLD.id_cliente;
    INSERT INTO "REPORTE_CLIENTES_ELIMINADOS" (id_cliente, doc_cliente, nombre_cliente, cantidad_cuentas) VALUES (OLD.id_cliente, v_documento, v_nombre, v_cant_cuentas);
    RETURN OLD;
END;
$$;

DROP TRIGGER IF EXISTS tr_auditar_eliminacion ON "CLIENTE";
CREATE TRIGGER tr_auditar_eliminacion BEFORE DELETE ON "CLIENTE" FOR EACH ROW EXECUTE FUNCTION auditar_eliminacion_cliente();
```

Listing 4: Tabla, función y trigger de auditoría de eliminación de clientes

9.4. D. Función fn_reporte_estadistico completa

```sql
-- Función alineada a la consulta estadística usada en src/main.py.
-- Devuelve exactamente los campos base y KPIs que consumen los reportes.

SET search_path TO "usb_bank";

CREATE OR REPLACE FUNCTION "usb_bank"."fn_reporte_estadistico"(
    p_fecha_inicio DATE DEFAULT NULL,
    p_fecha_fin DATE DEFAULT NULL
)
RETURNS TABLE (
    "ID_Cliente" INTEGER,
    "Titular" TEXT,
    "Tipo_Cliente" VARCHAR(1),
    "Canal_Onboarding" VARCHAR(1),
    "Total_Cuentas" BIGINT,
    "Total_Tarjetas" BIGINT,
    "Saldo_Total" NUMERIC,
```

```sql
    "Fecha_Registro" DATE,
    "Total_Clientes_Activos" BIGINT,
    "Total_Cuentas_Activas" BIGINT,
    "Total_Tarjetas_Activas" BIGINT,
    "Onboarding_Portal_Web" BIGINT,
    "Onboarding_App_Movil" BIGINT,
    "Onboarding_IVR_Otros" BIGINT,
    "Debe" NUMERIC,
    "Haber" NUMERIC
)
LANGUAGE SQL
STABLE
AS $$
    SELECT
        c.id_cliente AS "ID_Cliente",
        COALESCE(cn.primer_nombre || ' ' || cn.apellido, cj.nombre_org) AS "Titular",
        CASE
            WHEN c.tipo_cliente = 'Natural' THEN 'N'
            ELSE 'J' END AS "Tipo_Cliente",
        CASE
            WHEN can.descripcion ILIKE '%Web%' THEN 'W'
            WHEN can.descripcion ILIKE '%Movil%' THEN 'M'
            ELSE 'O'
        END AS "Canal_Onboarding",
        COALESCE(cu_agg.cuentas, 0) AS "Total_Cuentas",
        COALESCE(t_agg.tarjetas, 0) AS "Total_Tarjetas",
        COALESCE(cu_agg.saldo_total_cuenta, 0.00) AS "Saldo_Total",
        c.fecha_registro::DATE AS "Fecha_Registro",
        kpi.total_clientes_activos AS "Total_Clientes_Activos",
        kpi.total_cuentas_activas AS "Total_Cuentas_Activas",
        kpi.total_tarjetas_activas AS "Total_Tarjetas_Activas",
        kpi.onboarding_portal_web AS "Onboarding_Portal_Web",
        kpi.onboarding_app_movil AS "Onboarding_App_Movil",
        kpi.onboarding_ivr_otros AS "Onboarding_IVR_Otros",
        COALESCE(mov.total_ingresos, 0.00) AS "Debe",
        COALESCE(mov.total_egresos, 0.00) AS "Haber"
    FROM "usb_bank"."CLIENTE" c
    LEFT JOIN "usb_bank"."CLIENTE_NATURAL" cn ON c.id_cliente = cn.id_cliente
    LEFT JOIN "usb_bank"."CLIENTE_JURIDICO" cj ON c.id_cliente = cj.id_cliente
    JOIN "usb_bank"."CANAL" can ON c.id_canal_onboarding = can.id_canal
    LEFT JOIN (
        SELECT id_cliente,
               SUM(saldo) AS saldo_total_cuenta,
               COUNT(DISTINCT nro_cuenta) AS cuentas
        FROM "usb_bank"."CUENTA"
        GROUP BY id_cliente
    ) cu_agg ON c.id_cliente = cu_agg.id_cliente
    LEFT JOIN (
        SELECT cu.id_cliente,
               COUNT(DISTINCT t.nro_tarjeta) AS tarjetas
        FROM "usb_bank"."CUENTA" cu
        JOIN "usb_bank"."TARJETA" t ON cu.nro_cuenta = t.nro_cuenta
        GROUP BY cu.id_cliente
    ) t_agg ON c.id_cliente = t_agg.id_cliente
    CROSS JOIN (
        SELECT
            COUNT(DISTINCT c3.id_cliente) FILTER (WHERE LOWER(c3.estado) = 'activo') AS total_clientes_activos,
            (SELECT COUNT(*) FROM "usb_bank"."CUENTA" cu3 WHERE LOWER(cu3.estado) = 'activa') AS total_cuentas_activas,
            (SELECT COUNT(*) FROM "usb_bank"."TARJETA" t3 WHERE LOWER(t3.estado) = 'activa') AS total_tarjetas_activas,
            COUNT(DISTINCT c3.id_cliente) FILTER (WHERE LOWER(can3.descripcion) LIKE '%portal web%') AS onboarding_portal_web,
```

```
            COUNT(DISTINCT c3.id_cliente) FILTER (WHERE LOWER(can3.descripcion) LIKE '%app movil%') AS onboarding_app_movil,
            COUNT(DISTINCT c3.id_cliente) FILTER (WHERE LOWER(can3.descripcion) NOT LIKE '%portal web%' AND LOWER(can3.descripcion) NOT LIKE '%app movil%') AS onboarding_ivr_otros
        FROM "usb_bank"."CLIENTE" c3
        JOIN "usb_bank"."CANAL" can3 ON can3.id_canal = c3.id_canal_onboarding
        WHERE LOWER(c3.estado) = 'activo'
    ) kpi
    LEFT JOIN (
        WITH movimientos_filtrados AS (
            SELECT m.*
            FROM "usb_bank"."MOVIMIENTO" m
            WHERE (p_fecha_inicio IS NULL OR m.fecha >= p_fecha_inicio::timestamp)
              AND (p_fecha_fin IS NULL OR m.fecha < (p_fecha_fin + INTERVAL '1 day'))
        ),
        movimientos_unificados AS (
            SELECT m.nro_cuenta_origen AS nro_cuenta, m.monto_egreso AS egreso, 0.00::numeric AS ingreso FROM movimientos_filtrados m
            UNION ALL
            SELECT m.nro_cuenta_destino AS nro_cuenta, 0.00::numeric AS egreso, m.monto_ingreso AS ingreso FROM movimientos_filtrados m
        )
        SELECT cu_mov.id_cliente, COALESCE(SUM(mu.ingreso), 0.00) AS total_ingresos, COALESCE(SUM(mu.egreso), 0.00) AS total_egresos
        FROM movimientos_unificados mu
        JOIN "usb_bank"."CUENTA" cu_mov ON mu.nro_cuenta = cu_mov.nro_cuenta
        WHERE (mu.ingreso + mu.egreso) > 0
        GROUP BY cu_mov.id_cliente
    ) mov ON c.id_cliente = mov.id_cliente
    ORDER BY c.id_cliente;
$$;
```

Listing 5: Definición completa de fn_reporte_estadistico

9.5. E. Función fn_reporte_contable_base completa

```
-- Funciones SQL para reportes contables.
-- Estas funciones permiten ejecutar la logica de agregacion directamente en PostgreSQL.
-- Se pueden invocar desde psql o desde un procedimiento almacenado externo.

SET search_path TO "usb_bank";

-- Base reutilizable: normaliza movimientos y agrega campos de contexto.
CREATE OR REPLACE FUNCTION "usb_bank"."fn_reporte_contable_base"(
    p_fecha_inicio DATE DEFAULT NULL,
    p_fecha_fin DATE DEFAULT NULL,
    p_estado_movimiento TEXT DEFAULT 'Todos'
)
RETURNS TABLE (
    referencia VARCHAR(50),
    fecha_movimiento TIMESTAMP,
    estado_movimiento VARCHAR(20),
    direccion TEXT,
    nro_cuenta VARCHAR(20),
    id_cliente INTEGER,
    titular TEXT,
    estado_cliente VARCHAR(20),
    nro_cuentas_activas BIGINT,
    nro_tarjetas_activas BIGINT,
    tipo_cuenta VARCHAR(20),
    estado_cuenta VARCHAR(20),
```

```
    canal_descripcion VARCHAR(100),
    onboardings_canal BIGINT,
    nro_tarjeta VARCHAR(16),
    tipo_tarjeta VARCHAR(20),
    estado_tarjeta VARCHAR(20),
    tipo_movimiento VARCHAR(100),
    ingreso NUMERIC,
    egreso NUMERIC,
    comision NUMERIC,
    neto NUMERIC,
    monto_movido NUMERIC,
    total_clientes_activos_bbdd BIGINT,
    total_cuentas_activas_bbdd BIGINT,
    total_tarjetas_activas_bbdd BIGINT
)
LANGUAGE SQL
STABLE
AS $$
    WITH estado_normalizado AS (
        SELECT CASE
            WHEN p_estado_movimiento IS NULL OR BTRIM(p_estado_movimiento) = '' THEN 'Todos'
            WHEN LOWER(BTRIM(p_estado_movimiento)) LIKE 'estado %' THEN SUBSTRING(LOWER(BTRIM(p_estado_movimiento)) FROM 8)
            ELSE LOWER(BTRIM(p_estado_movimiento))
        END AS estado_filtro
    ), movimientos_filtrados AS (
        SELECT m.*
        FROM "usb_bank"."MOVIMIENTO" m
        WHERE (p_fecha_inicio IS NULL OR m.fecha >= p_fecha_inicio::timestamp)
          AND (p_fecha_fin IS NULL OR m.fecha < (p_fecha_fin + INTERVAL '1 day'))
                    AND (
                                LOWER((SELECT estado_filtro FROM estado_normalizado)) = 'todos'
                                OR LOWER(m.estado) = LOWER((SELECT estado_filtro FROM estado_normalizado))
                            )
    ),
    movimientos_unificados AS (
        SELECT
            m.nro_referencia,
            m.fecha,
            m.estado,
            m.id_canal,
            m.id_tipo_mov,
            m.id_banco_origen,
            m.id_banco_destino,
            m.descripcion_mov,
            m.monto_comision,
            m.nro_cuenta_origen AS nro_cuenta,
            'egreso'::text AS direccion,
            m.monto_egreso AS egreso,
            0.00::numeric AS ingreso,
            m.saldo_origen_previo AS saldo_previo,
            m.saldo_origen_nuevo AS saldo_nuevo
        FROM movimientos_filtrados m

        UNION ALL

        SELECT
            m.nro_referencia,
            m.fecha,
            m.estado,
            m.id_canal,
            m.id_tipo_mov,
```

```sql
            m.id_banco_origen,
            m.id_banco_destino,
            m.descripcion_mov,
            0.00::numeric AS monto_comision,
            m.nro_cuenta_destino AS nro_cuenta,
            'ingreso'::text AS direccion,
            0.00::numeric AS egreso,
            m.monto_ingreso AS ingreso,
            m.saldo_destino_previo AS saldo_previo,
            m.saldo_destino_nuevo AS saldo_nuevo
        FROM movimientos_filtrados m
    )
    SELECT
        mu.nro_referencia AS referencia,
        mu.fecha AS fecha_movimiento,
        mu.estado AS estado_movimiento,
        mu.direccion,
        mu.nro_cuenta,
        cu.id_cliente,
        COALESCE(cn.primer_nombre || ' ' || cn.apellido, cj.nombre_org, 'SIN TITULAR') AS titular,
        c.estado AS estado_cliente,
        COALESCE(cc.nro_cuentas_activas, 0) AS nro_cuentas_activas,
        COALESCE(tc.nro_tarjetas_activas, 0) AS nro_tarjetas_activas,
        cu.tipo_cuenta,
        cu.estado AS estado_cuenta,
        can_onb.descripcion AS canal_descripcion,
        COALESCE(ob.nro_onboardings, 0) AS onboardings_canal,
        COALESCE(pos.nro_tarjeta, ecom.nro_tarjeta, atm.nro_tarjeta, 'N/A') AS nro_tarjeta,
        COALESCE(t.tipo_tarjeta, 'N/A') AS tipo_tarjeta,
        COALESCE(t.estado, 'N/A') AS estado_tarjeta,
        tm.descripcion AS tipo_movimiento,
        mu.ingreso,
        mu.egreso,
        mu.monto_comision AS comision,
        (mu.ingreso - mu.egreso) AS neto,
        (mu.ingreso + mu.egreso) AS monto_movido,
        totales.total_clientes_activos,
        totales.total_cuentas_activas,
        totales.total_tarjetas_activas
    FROM movimientos_unificados mu
    JOIN "usb_bank"."CUENTA" cu ON mu.nro_cuenta = cu.nro_cuenta
    JOIN "usb_bank"."CLIENTE" c ON cu.id_cliente = c.id_cliente
    LEFT JOIN "usb_bank"."CLIENTE_NATURAL" cn ON c.id_cliente = cn.id_cliente
    LEFT JOIN "usb_bank"."CLIENTE_JURIDICO" cj ON c.id_cliente = cj.id_cliente
    LEFT JOIN (
        SELECT id_cliente, COUNT(DISTINCT nro_cuenta) AS nro_cuentas_activas
        FROM "usb_bank"."CUENTA"
        WHERE LOWER(estado) = 'activa'
        GROUP BY id_cliente
    ) cc ON cu.id_cliente = cc.id_cliente
    LEFT JOIN (
        SELECT cu2.id_cliente, COUNT(DISTINCT t2.nro_tarjeta) AS nro_tarjetas_activas
        FROM "usb_bank"."CUENTA" cu2
        LEFT JOIN "usb_bank"."TARJETA" t2
          ON cu2.nro_cuenta = t2.nro_cuenta
         AND LOWER(t2.estado) = 'activa'
        GROUP BY cu2.id_cliente
    ) tc ON cu.id_cliente = tc.id_cliente
    LEFT JOIN (
        SELECT id_canal_onboarding AS id_canal, COUNT(DISTINCT id_cliente) AS nro_onboardings
        FROM "usb_bank"."CLIENTE"
        WHERE LOWER(estado) = 'activo'
```

```
        GROUP BY id_canal_onboarding
    ) ob ON c.id_canal_onboarding = ob.id_canal
    CROSS JOIN (
        SELECT
            COUNT(DISTINCT c3.id_cliente) FILTER (WHERE LOWER(c3.estado) = 'activo') AS total_clientes_activos,
            (SELECT COUNT(*) FROM "usb_bank"."CUENTA" cu3 WHERE LOWER(cu3.estado) = 'activa') AS total_cuentas_activas,
            (SELECT COUNT(*) FROM "usb_bank"."TARJETA" t3 WHERE LOWER(t3.estado) = 'activa') AS total_tarjetas_activas
        FROM "usb_bank"."CLIENTE" c3
    ) totales
    JOIN "usb_bank"."CANAL" can_onb ON c.id_canal_onboarding = can_onb.id_canal
    JOIN "usb_bank"."TIPO_MOVIMIENTO" tm ON mu.id_tipo_mov = tm.id_tipo_mov
    LEFT JOIN "usb_bank"."MOVIMIENTO_PAGO_POS" pos ON mu.nro_referencia = pos.nro_referencia
    LEFT JOIN "usb_bank"."MOVIMIENTO_PAGO_ECOMMERCE" ecom ON mu.nro_referencia = ecom.nro_referencia
    LEFT JOIN "usb_bank"."MOVIMIENTO_RETIRO_ATM" atm ON mu.nro_referencia = atm.nro_referencia
    LEFT JOIN "usb_bank"."TARJETA" t ON t.nro_tarjeta = COALESCE(pos.nro_tarjeta, ecom.nro_tarjeta, atm.nro_tarjeta)
    WHERE (mu.ingreso + mu.egreso) > 0;
$$;
```

Listing 6: Definición completa de fn_reporte_contable_base

9.6. F. Extracto de sp_imprimir_reporte_estadistico (RAISE NOTICE)

```
    RAISE NOTICE '%', linea_separadora;
    RAISE NOTICE '| % | % | % | % | % | % | % | % | % | % |',
        RPAD('ID', ancho_id),
        RPAD('Titular', ancho_titular),
        RPAD('Tipo Cliente', ancho_tipo),
        RPAD('Canal Onboarding', ancho_canal),
        LPAD('Total Cuentas', ancho_cuentas),
        LPAD('Total Tarjetas', ancho_tarjetas),
        RPAD('Fecha Registro', ancho_fecha),
        LPAD('Debe', ancho_debe),
        LPAD('Haber', ancho_haber),
        LPAD('Saldo Total', ancho_saldo);
    RAISE NOTICE '%', linea_separadora;

    FOR registro IN (
        SELECT *
        FROM "usb_bank"."fn_reporte_estadistico"(p_fecha_inicio, p_fecha_fin)
                WHERE (LOWER(COALESCE(v_tipo_cliente_filtro, 'Todos')) = 'todos' OR "Tipo_Cliente" = v_tipo_cliente_filtro)
                    AND (LOWER(COALESCE(v_canal_onboarding_filtro, 'Todos')) = 'todos' OR "Canal_Onboarding" = v_canal_onboarding_filtro)
        ORDER BY "ID_Cliente"
    ) LOOP
        v_hay_datos := TRUE;
        v_total_ingresos := v_total_ingresos + COALESCE(registro."Debe", 0);
        v_total_egresos := v_total_egresos + COALESCE(registro."Haber", 0);
        v_total_clientes_activos := registro."Total_Clientes_Activos";
        v_total_cuentas_activas := registro."Total_Cuentas_Activas";
        v_total_tarjetas_activas := registro."Total_Tarjetas_Activas";
        v_onboarding_portal_web := registro."Onboarding_Portal_Web";
        v_onboarding_app_movil := registro."Onboarding_App_Movil";
```

```

        RAISE NOTICE '| % | % | % | % | % | % | % | % | % | % |',
            LPAD(registro."ID_Cliente"::TEXT, ancho_id),
            RPAD(COALESCE(SUBSTRING(registro."Titular", 1, ancho_titular), ''), ancho_titular),
            RPAD(COALESCE(registro."Tipo_Cliente", ''), ancho_tipo),
            RPAD(COALESCE(registro."Canal_Onboarding", ''), ancho_canal),
            LPAD(COALESCE(registro."Total_Cuentas", 0)::TEXT, ancho_cuentas),
            LPAD(COALESCE(registro."Total_Tarjetas", 0)::TEXT, ancho_tarjetas),
            RPAD(COALESCE(TO_CHAR(registro."Fecha_Registro", 'YYYY-MM-DD'), ''), ancho_fecha),
            LPAD(REPLACE(REPLACE(REPLACE(TO_CHAR(COALESCE(registro."Debe", 0), 'FM999,999,999,990.00'), ',', '_'), '.', ','), '_', '.'), ancho_debe),
            LPAD(REPLACE(REPLACE(REPLACE(TO_CHAR(COALESCE(registro."Haber", 0), 'FM999,999,999,990.00'), ',', '_'), '.', ','), '_', '.'), ancho_haber),
            LPAD(REPLACE(REPLACE(REPLACE(TO_CHAR(COALESCE(registro."Saldo_Total", 0), 'FM999,999,999,990.00'), ',', '_'), '.', ','), '_', '.'), ancho_saldo);
    END LOOP;

    RAISE NOTICE ' %', linea_separadora;

    IF NOT v_hay_datos THEN
        SELECT
            COUNT(DISTINCT c.id_cliente) FILTER (WHERE LOWER(c.estado) = 'activo'),
            (SELECT COUNT(*) FROM "usb_bank"."CUENTA" cu WHERE LOWER(cu.estado) = 'activa'),
            (SELECT COUNT(*) FROM "usb_bank"."TARJETA" t WHERE LOWER(t.estado) = 'activa'),
            COUNT(DISTINCT c.id_cliente) FILTER (WHERE LOWER(can.descripcion) LIKE '%portal web%'),
            COUNT(DISTINCT c.id_cliente) FILTER (WHERE LOWER(can.descripcion) LIKE '%app movil%')
        INTO
            v_total_clientes_activos,
            v_total_cuentas_activas,
            v_total_tarjetas_activas,
            v_onboarding_portal_web,
            v_onboarding_app_movil
        FROM "usb_bank"."CLIENTE" c
        JOIN "usb_bank"."CANAL" can ON can.id_canal = c.id_canal_onboarding
        WHERE LOWER(c.estado) = 'activo';
    END IF;

    v_balance_neto := v_total_ingresos - v_total_egresos;

    v_rango_fechas := CASE
        WHEN p_fecha_inicio IS NULL AND p_fecha_fin IS NULL THEN 'Todos los registros hasta la fecha'
        ELSE FORMAT(
            'Desde %s Hasta %s',
            COALESCE(TO_CHAR(p_fecha_inicio, 'YYYY-MM-DD'), 'N/A'),
            COALESCE(TO_CHAR(p_fecha_fin, 'YYYY-MM-DD'), 'N/A')
        )
    END;

    RAISE NOTICE '';
    RAISE NOTICE 'Resumen de Operaciones (%)', v_rango_fechas;
    RAISE NOTICE '-----------------------------------------------------------';
    RAISE NOTICE 'TOTAL GENERAL DE INGRESOS (DEBE): VES %',
        REPLACE(REPLACE(REPLACE(TO_CHAR(COALESCE(v_total_ingresos, 0), 'FM999,999,999,990.00'), ',', '_'), '.', ','), '_', '.');
    RAISE NOTICE 'TOTAL GENERAL DE EGRESOS (HABER): VES %',
        REPLACE(REPLACE(REPLACE(TO_CHAR(COALESCE(v_total_egresos, 0), 'FM999,999,999,990.00'), ',', '_'), '.', ','), '_', '.');
    RAISE NOTICE 'BALANCE NETO DEL PERIODO: VES %',
```

```
        REPLACE(REPLACE(REPLACE(TO_CHAR(COALESCE(v_balance_neto, 0), 'FM999,999,999,990.00'), ',', '_'), '.', ','), '_', '.');

    RAISE NOTICE '';
    RAISE NOTICE 'Información del Sistema';
    RAISE NOTICE '-----------------------';
    RAISE NOTICE 'TOTAL CLIENTES ACTIVOS: %', COALESCE(v_total_clientes_activos, 0);
    RAISE NOTICE 'TOTAL CUENTAS ACTIVAS: %', COALESCE(v_total_cuentas_activas, 0);
    RAISE NOTICE 'TOTAL TARJETAS ACTIVAS: %', COALESCE(v_total_tarjetas_activas, 0);
    RAISE NOTICE 'Onboarding por Canal - Portal Web: % clientes', COALESCE(v_onboarding_portal_web, 0);
    RAISE NOTICE 'Onboarding por Canal - App Móvil: % clientes', COALESCE(v_onboarding_app_movil, 0);
```

Listing 7: Extracto con salida tabular y resumen usando RAISE NOTICE

9.7. G. Python: conexión PostgreSQL y obtener_datos_y_generar (main.py)

```python
import psycopg2
import pandas as pd
import warnings

from config import config
from generador_reportes import (
    generador_reportes_estadisticos,
    generador_reportes_contables,
    generador_reportes_auditoria,
)

warnings.filterwarnings("ignore", category=UserWarning)


def normalizar_estado_movimiento(estado_raw):
    """Normaliza etiquetas de estado provenientes de GUI/entrada libre."""
    estado = (estado_raw or "Todos").strip()
    if not estado:
        return "Todos"

    estado_lower = estado.lower()
    if estado_lower.startswith("estado "):
        estado = estado[7:].strip() or "Todos"
        estado_lower = estado.lower()

    etiquetas = {
        "todos": "Todos",
        "completado": "Completado",
        "pendiente": "Pendiente",
        "fallido": "Fallido",
        "reversado": "Reversado",
    }
    return etiquetas.get(estado_lower, estado)


def obtener_datos_y_generar(tipo_reporte, filtros):
    connection = None

    try:
        params = config()
        print("Conectando con la base de datos en PostgreSQL ...")
        connection = psycopg2.connect(**params)
```

```
        cursor = connection.cursor()

        query = None
        query_params = None

        if tipo_reporte == "estadistico":
            fecha_inicio = filtros.get("inicio") if filtros.get("inicio") else None
            fecha_fin = filtros.get("fin") if filtros.get("fin") else None

            query = 'SELECT * FROM "usb_bank"."fn_reporte_estadistico"(%s, %s);'
            query_params = (fecha_inicio, fecha_fin)

        elif tipo_reporte == "contable":
            fecha_inicio = filtros.get("inicio") if filtros.get("inicio") else None
            fecha_fin = filtros.get("fin") if filtros.get("fin") else None
            estado_movimiento = normalizar_estado_movimiento(
                filtros.get("estado_movimiento")
            )

            query = (
                'SELECT * FROM "usb_bank"."fn_reporte_contable_detalle"(%s, %s, %s);'
            )
            query_params = (fecha_inicio, fecha_fin, estado_movimiento)

        elif tipo_reporte == "auditoria":
            print("Iniciando rutina de limpieza de inactivos...")

            query_delete = """
                SET search_path TO "usb_bank";

                DELETE FROM "usb_bank"."CLIENTE" c
                WHERE c.id_cliente IN (
                    SELECT cu.id_cliente
                    FROM "usb_bank"."CUENTA" cu
                    JOIN "usb_bank"."TARJETA" t ON cu.nro_cuenta = t.nro_cuenta
                )
                AND c.id_cliente NOT IN (
                    SELECT cu.id_cliente
                    FROM "usb_bank"."CUENTA" cu
                    JOIN "usb_bank"."TARJETA" t ON cu.nro_cuenta = t.nro_cuenta
                    JOIN "usb_bank"."MOVIMIENTO_PAGO_POS" mp ON t.nro_tarjeta = mp.nro_tarjeta
                    JOIN "usb_bank"."MOVIMIENTO" m ON mp.nro_referencia = m.nro_referencia
                    WHERE m.fecha >= CURRENT_DATE - INTERVAL '1 month'

                    UNION

                    SELECT cu.id_cliente
                    FROM "usb_bank"."CUENTA" cu
                    JOIN "usb_bank"."TARJETA" t ON cu.nro_cuenta = t.nro_cuenta
                    JOIN "usb_bank"."MOVIMIENTO_PAGO_ECOMMERCE" me ON t.nro_tarjeta = me.nro_tarjeta
                    JOIN "usb_bank"."MOVIMIENTO" m ON me.nro_referencia = m.nro_referencia
                    WHERE m.fecha >= CURRENT_DATE - INTERVAL '1 month'

                    UNION

                    SELECT cu.id_cliente
                    FROM "usb_bank"."CUENTA" cu
                    JOIN "usb_bank"."TARJETA" t ON cu.nro_cuenta = t.nro_cuenta
                    JOIN "usb_bank"."MOVIMIENTO_RETIRO_ATM" ma ON t.nro_tarjeta = ma.nro_tarjeta
```

```
                    JOIN "usb_bank"."MOVIMIENTO" m ON ma.nro_referencia = m.nro_referencia
                    WHERE m.fecha >= CURRENT_DATE - INTERVAL '1 month'
                );
            """

            cursor = connection.cursor()
            cursor.execute(query_delete)
            eliminados = cursor.rowcount
            connection.commit()
            cursor.close()
            print(
                f"Limpieza ejecutada. Se eliminaron {eliminados} clientes. Triggers disparados."
            )

            query = 'SELECT * FROM "usb_bank"."REPORTE_CLIENTES_ELIMINADOS" ORDER BY fecha_eliminacion DESC;'

        print(f"Descargando datos de PostgreSQL para reporte {tipo_reporte} ...")
        df = pd.read_sql_query(query, connection, params=query_params)

        if "ID" in df.columns and "ID_Cliente" not in df.columns:
            df.rename(columns={"ID": "ID_Cliente"}, inplace=True)

        formato = filtros.get("formato_deseado", "ambos")

        resultado_generador = None

        if tipo_reporte == "estadistico":
            resultado_generador = generador_reportes_estadisticos(
                df_clientes=df,
                fecha_inicio=filtros.get("inicio"),
                fecha_fin=filtros.get("fin"),
                tipo_cliente=filtros.get("tipo"),
                canal=filtros.get("canal"),
                formato=formato,
            )
        elif tipo_reporte == "contable":
            resultado_generador = generador_reportes_contables(
                df_movimientos=df,
                agrupar_por=filtros.get("agrupar_por", "cliente"),
                formato=formato,
                fecha_inicio=filtros.get("inicio"),
                fecha_fin=filtros.get("fin"),
                estado_movimiento=normalizar_estado_movimiento(
                    filtros.get("estado_movimiento", "Todos")
                ),
            )
        elif tipo_reporte == "auditoria":
            resultado_generador = generador_reportes_auditoria(
                df_auditoria=df, formato=formato
            )

        if isinstance(resultado_generador, dict):
            if not resultado_generador.get("has_data", True):
                mensaje_sin_datos = (
                    "Reporte creado pero no hay datos de movimientos para generar este reporte."
                    if tipo_reporte == "contable"
                    else "Reporte creado pero no hay información con los filtros seleccionados."
                )
                return {
                    "ok": True,
```

```
                    "has_data": False,
                    "message": mensaje_sin_datos,
                }

            return {
                "ok": True,
                "has_data": True,
                "message": "Reporte creado con éxito.",
            }

        return {
            "ok": True,
            "has_data": True,
            "message": "Reporte creado con éxito.",
        }

    except Exception as error:
        print(f"Error critico en el proceso: {error}")
        if connection:
            connection.rollback()
        raise
    finally:
        if connection is not None:
            connection.close()
            print("Conexión cerrada.")
```

Listing 8: Extracto de main.py con conexión a PostgreSQL y orquestación de reportes

9.8. H. Python: extracto clave de generador_reportes.py

```
def exportar_pdf(
    datos,
    nombre_reporte,
    subcarpeta,
    resumen_tabla=None,
    metadata_lineas=None,
    secciones_resumen=None,
    saldo_total_texto=None,
    anchos_columnas=None,
):
    def texto_pdf(valor):
        if pd.isna(valor):
            return ""
        return str(valor)

    columnas = [str(col) for col in datos.columns.tolist()]
    columnas_pdf = normalizar_nombres_columnas_visuales(columnas, subcarpeta)
    subcarpetas_formato_corporativo = {
        "reportes_contables",
        "reportes_estadisticos",
        "reportes_auditoria",
    }
    orientacion = (
        "L"
        if (subcarpeta in subcarpetas_formato_corporativo or len(columnas) > 7)
        else "P"
    )
    etiqueta_reporte = obtener_etiqueta_reporte(subcarpeta)

    pdf = PDFReporte(orientacion=orientacion, etiqueta_reporte=etiqueta_reporte)
    pdf.add_page()

```

```
    pdf.set_line_width(0.2)

    # Encabezado del pdf
    pdf.set_font("helvetica", style="B", size=16)
    pdf.cell(
        0,
        10,
        f"USB Bank - {etiqueta_reporte}",
        align="C",
        new_x="LMARGIN",
        new_y="NEXT",
    )

    # Fecha del pdf
    pdf.set_font("helvetica", style="I", size=10)
    fecha_actual = datetime.now().strftime("%d/%m/%Y  %H:%M:%S")
    pdf.cell(
        0,
        10,
        f"Fecha de emisión: {fecha_actual}",
        align="C",
        new_x="LMARGIN",
        new_y="NEXT",
    )
    pdf.ln(5)

    filas = datos.values.tolist()

    # dibujo la tabla especificada
    tam_fuente_tabla = 8 if len(columnas) > 10 else 9
    pdf.set_font("helvetica", size=tam_fuente_tabla)
    encabezado_style = (
        FontFace(emphasis="B", fill_color=PASTEL_BLUE)
        if subcarpeta in subcarpetas_formato_corporativo
        else FontFace(emphasis="B")
    )
    with pdf.table(
        text_align="CENTER", headings_style=encabezado_style, col_widths=anchos_columnas
    ) as table:
        encabezado = table.row()
        for col_name in columnas_pdf:
            encabezado.cell(col_name)

        pdf.set_font(style="")
        for fila_datos in filas:
            fila = table.row()
            for item in fila_datos:
                fila.cell(texto_pdf(item))

    if subcarpeta == "reportes_estadisticos" and saldo_total_texto:
        pdf.ln(5)
        pdf.set_font("helvetica", style="I", size=10)
        pdf.cell(
            0,
            10,
            f"Saldo Total: {saldo_total_texto}\n",
            align="L",
            new_x="LMARGIN",
            new_y="NEXT",
        )

    if resumen_tabla:
        pdf.ln(4)
        pdf.set_font("helvetica", style="B", size=11)
        pdf.cell(0, 8, "Resumen del Reporte", new_x="LMARGIN", new_y="NEXT")
```

```python

        pdf.set_font("helvetica", size=10)
        page_width_usable = pdf.w - pdf.l_margin - pdf.r_margin
        label_w = page_width_usable * 0.7
        value_w = page_width_usable * 0.3
        for etiqueta, valor in resumen_tabla:
            pdf.set_fill_color(*PASTEL_BLUE)
            pdf.cell(label_w, 8, texto_pdf(etiqueta), border=1, align="L", fill=True)
            pdf.cell(value_w, 8, texto_pdf(valor), border=1, align="R", fill=True)
            pdf.ln(8)

    if secciones_resumen:
        for seccion in secciones_resumen:
            titulo = seccion.get("titulo", "")
            filas_seccion = seccion.get("filas", [])
            if not filas_seccion:
                continue

            pdf.ln(4)
            pdf.set_font("helvetica", style="B", size=11)
            pdf.cell(0, 8, str(titulo), new_x="LMARGIN", new_y="NEXT")

            pdf.set_font("helvetica", size=10)
            page_width_usable = pdf.w - pdf.l_margin - pdf.r_margin
            label_w = page_width_usable * 0.7
            value_w = page_width_usable * 0.3

            for etiqueta, valor in filas_seccion:
                pdf.set_fill_color(*PASTEL_BLUE)
                pdf.cell(
                    label_w, 8, texto_pdf(etiqueta), border=1, align="L", fill=True
                )
                pdf.cell(value_w, 8, texto_pdf(valor), border=1, align="R", fill=True)
                pdf.ln(8)

    if metadata_lineas:
        pdf.ln(4)
        pdf.set_font("helvetica", style="B", size=10)
        pdf.cell(0, 8, "Parametros del Reporte", new_x="LMARGIN", new_y="NEXT")
        pdf.set_font("helvetica", size=10)
        for linea in metadata_lineas:
            pdf.cell(0, 7, texto_pdf(linea), new_x="LMARGIN", new_y="NEXT")

    # Guardo el archivo
    if not nombre_reporte.endswith(".pdf"):
        nombre_reporte += ".pdf"

    ruta_destino = os.path.join("reportes_generados", subcarpeta, nombre_reporte)
    pdf.output(ruta_destino)
    print(f"[PDF OK] Reporte guardado en: {ruta_destino}")


# generador_reportes_estadisticos genera los reportes estadisticos con los requerimientos
    ↪ solicitados
```

Listing 9: Extracto de exportación PDF y estructura del generador de reportes

10. Referencias

[1] IEEE Computer Society, *IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*, 2009.

[2] R. Elmasri, S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016. (Cap. 14: Normalización)

[3] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024. `https://www.postgresql.org/docs/16/`

[4] Universidad Simón Bolívar, *Guía de Proyecto CI3391 - Fase III*, Departamento de Computación y TI, Trimestre I-2026.